STM32 MICROCONTROLLER COURSE

CRC Calculation Unit

Hardware-Accelerated Cyclic Redundancy Check on STM32

GROUP 9

2210776130	**Md Shibli Noman Sarkar**
2210776147	**Md Ruhul Amin**
2210776132	**Md Mehedi Hasan Rafy**

What Is CRC?

Definition

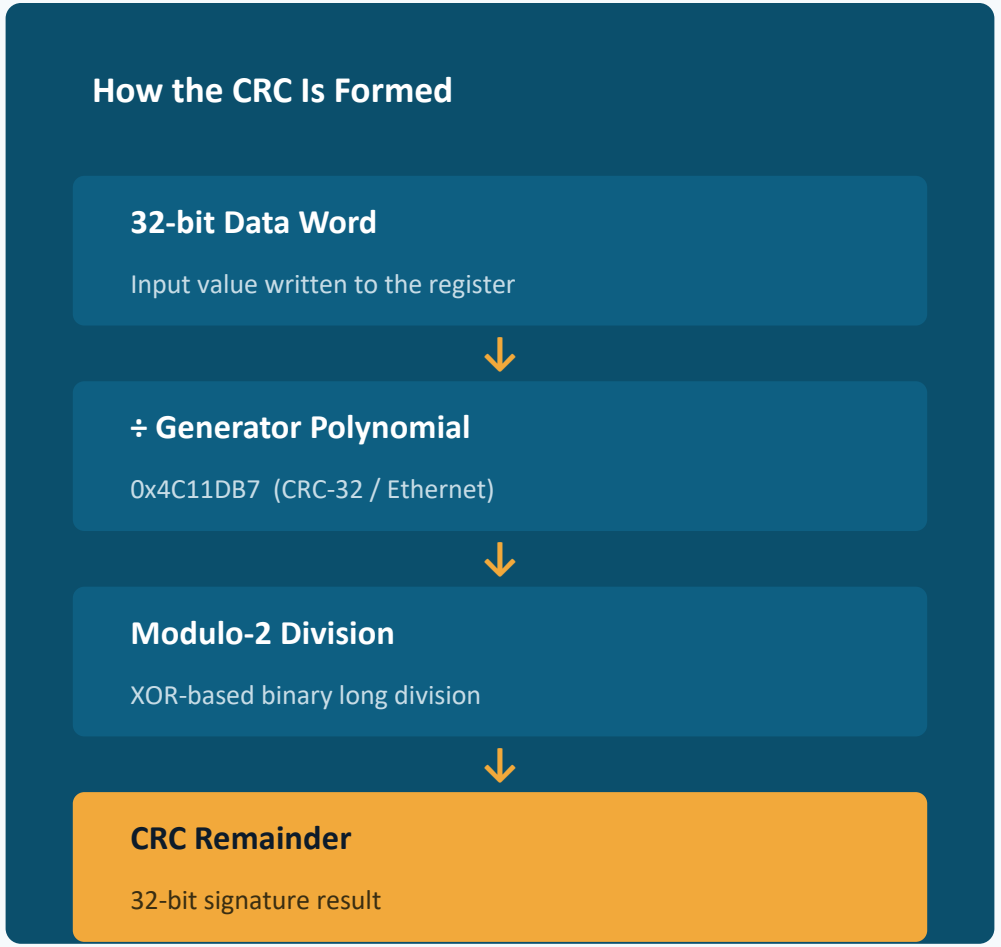
The CRC calculation unit derives a fixed-length CRC code from a 32-bit data word using a fixed generator polynomial — a hardware checksum engine built into the STM32.

The Theory

- Data is treated as a binary polynomial and divided by a fixed generator polynomial using modulo-2 (XOR) arithmetic.
- The remainder of that division is the CRC signature — a compact fingerprint of the data.
- Any bit change in the data almost certainly changes the remainder, so mismatched CRCs reveal corruption.

Why it matters

Verifies data transmission & storage integrity, and checks flash memory contents against the EN/IEC 60335-1 safety standard.



CRC Main Features & Block Diagram

1 **Polynomial**

Uses the CRC-32 (Ethernet) polynomial: 0x4C11DB7

2 **Registers**

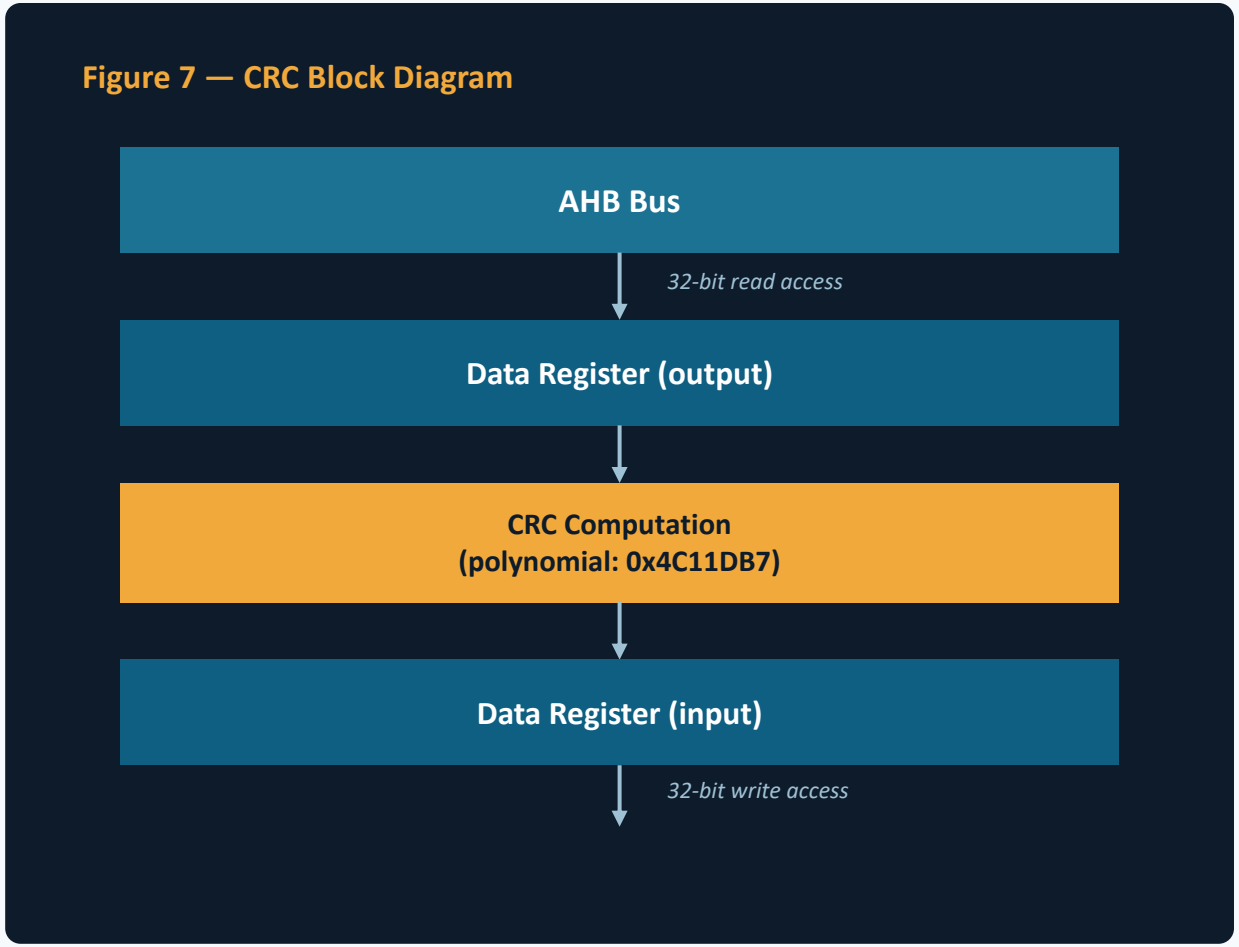
Single 32-bit input/output data register

3 **Speed**

CRC computation completed in just 4 AHB clock cycles (HCLK)

4 **Bonus Storage**

Extra general-purpose 8-bit register for temporary data



How the CRC Unit Operates

The CRC calculation unit mainly consists of a single 32-bit data register, which is used to enter new data and holds the result of the previous calculation when read.

1

Write to Combine

Every write into the data register combines the previous CRC value with the new data — the whole 32-bit word is processed at once, not byte-by-byte.

2

Stall Until Done

A write is stalled until CRC computation finishes, so back-to-back writes or a write followed by a read work safely without extra handshaking.

3

Reset to 0xFFFFFFFF

Setting the RESET bit in CRC_CR reinitializes the calculator to 0xFFFF FFFF. The independent CRC_IDR register is left untouched.

Note: The CRC unit contains two data registers and a control register — all accessed by full 32-bit words.

Data Register & Independent Data Register

CRC_DR — Data Register

Offset 0x00

Reset value: 0xFFFF FFFF



rw · read/write, all 32 bits

Function

- Used as the input register to enter new data into the CRC calculator on write.
- Holds the result of the previous CRC calculation when read back.

CRC_IDR — Independent Reg.

Offset 0x04

Reset value: 0x0000 0000



rw · only bits 7:0 usable

Function

- General-purpose 8-bit register — free temporary byte storage, unrelated to the CRC math.
- Not affected by a CRC RESET, so scratch data survives a CRC recalculation.

Control Register & Register Map

CRC_CR — Control Register

Offset 0x08

Reset value: 0x0000 0000

Reserved [31:1]	RESET

Bit 0 — RESET

- Resets the CRC calculation unit and sets the data register to 0xFFFF FFFF.
- Write-only: this bit can only be set by software; hardware clears it automatically.

Table 13 — Register Map & Reset Values

Offset	Register	Reset Value
0x00	CRC_DR — Data register	0xFFFF FFFF
0x04	CRC_IDR — Independent data register	0x0000 0000
0x08	CRC_CR — Control register	0x0000 0000

Access Rule

All three CRC registers must be accessed as full 32-bit words — the peripheral does not support byte or half-word access. Reserved bits always read as 0 and must be kept at their reset value on writes.

STM32's Role in Data Integrity

STM32 doesn't just support CRC in software — it embeds a dedicated hardware CRC-32 engine directly on the AHB bus, turning a slow software task into a near-instant peripheral operation.

4

AHB CLOCK CYCLES

to compute a full 32-bit CRC in hardware — versus hundreds of CPU cycles for an equivalent software routine.

1

Firmware Integrity

Computes a runtime signature of the software, compared against a reference generated at link-time and stored in memory.

2

Safety Compliance

Provides the flash-memory integrity check required by the EN/IEC 60335-1 household-appliance safety standard.

3

CPU Offload

Frees the processor core entirely — the CRC engine runs independently on the AHB bus while the CPU does other work.

4

Communication Checks

Validates data received over UART, CAN, SPI, or storage links, catching transmission errors before they matter.

Thank You

Questions & Discussion Welcome

GROUP 9

Md Shibli Noman Sarkar · Md Ruhul Amin · Md Mehedi Hasan Rafy